Virtual Environments

Comprehensive Guide: venv, Conda & Poetry

Agenda

- Introduction - Why Environment Management?
- Python venv - Built-in Virtual Environments
- Conda - The Data Science Powerhouse
- Poetry - Modern Dependency Management (v2.3.2)
- Tool Comparison

Why Environment Management?

The Problem:

- Dependency Conflicts: Project A needs pandas 1.5, Project B needs pandas 2.0
- Reproducibility Issues: Code works on your machine but fails elsewhere
- System Pollution: Global Python cluttered with hundreds of packages
- Version Incompatibilities: Package X requires Python 3.8, Package Y needs 3.11

Solution: Isolated environments for each project!

Three Main Environment Tools

venv

(Python Built-in)

- Lightweight
- Included with Python 3.3+
- Simple projects

Conda

(Full-featured)

- Python + non-Python deps
- Data science focused
- Pre-compiled binaries

Poetry

(Modern v2.3)

- Deterministic builds
- Lock files
- Package development

Python venv: Built-in Virtual Environments

What is venv?

- Part of Python standard library (Python 3.3+)
- Creates isolated Python interpreter and package directories
- Lightweight - only copies/symlinks necessary files
- Each environment has independent site-packages
- Inherits Python version from system Python

When to use: Quick projects, learning Python, web development

venv: Creating Environment

Basic Command:

```
python -m venv myenv
```

Command Breakdown:

python - Python interpreter (use python3 on some systems)
-m - Run library module as script (m = module)
venv - The venv module name
myenv - Directory name for the environment (environment name)

venv: Folder Structure

What Gets Created:

```
myenv/
├── bin/                # Unix executables
│   ├── activate        # Activation script
│   ├── pip            # Package installer
│   └── python         # Python interpreter
├── Scripts/            # Windows executables
│   ├── activate.bat # CMD activation
│   ├── Activate.ps1 # PowerShell activation
│   └── python.exe    # Python interpreter
├── lib/                # Installed packages
│   └── python3.x/site-packages/
├── include/            # C headers
└── pyvenv.cfg          # Environment config
```

Key Files Explained:

- pyvenv.cfg - Configuration with Python home path
- activate - Script to activate the environment
- site-packages/ - Where pip installs packages

venv: Activation Workflow

Platform-Specific Activation:

```
# Windows Command Prompt
myenv\Scripts\activate.bat

# Linux / macOS
source myenv/bin/activate
```

After Activation:

```
(myenv) user@machine:~$  # Prompt shows environment name
```

Deactivation:

```
deactivate  # Works on all platforms
```

venv: requirements.txt Workflow

Creating requirements.txt:

```
# Export all installed packages
pip freeze > requirements.txt
# > redirects output to file
```

Sample requirements.txt:

```
numpy==1.24.3
pandas==2.0.2
scikit-learn>=1.3.0
matplotlib~=3.7.0
pytest==7.4.0
```

Installing from requirements.txt:

```
pip install -r requirements.txt
# -r = requirements file
```

Version Specifiers:

==1.2.3 → Exact version
>=1.2.0 → Minimum version
~=1.2.0 → Compatible release (>=1.2.0, <1.3.0)
>=1.0,<2.0 → Version range

venv: Common pip Commands

Essential pip Commands:

```
pip install numpy                       # Install latest version
pip install pandas==1.5.0               # Install specific version
pip install numpy pandas scikit-learn   # Install multiple packages
pip install -U numpy                    # Update package (-U = --upgrade)
pip uninstall numpy                     # Remove package

pip list                                # Show all installed packages
pip show numpy                          # Show package details

pip install --upgrade pip               # Upgrade pip itself
pip freeze                              # List installed packages with versions
pip freeze > requirements.txt           # Export to file
```

venv Workflow: Creating & Using venv

Complete Workflow:

```
# 1. Create virtual environment
python -m venv myproject_env

# 2. Activate it
source myproject_env/bin/activate  # Linux/Mac
myproject_env\Scripts\activate    # Windows

# 3. Upgrade pip (best practice)
pip install --upgrade pip

# 4. Install packages
pip install pandas numpy matplotlib scikit-learn

# 5. Save dependencies
pip freeze > requirements.txt

# 6. Work on your project...

# 7. Deactivate when done
deactivate
```

venv Workflow: Sharing with Team Members

What to Share:

- requirements.txt file (contains all package versions)
- README with Python version requirement
- Your project source code
- DO NOT commit the venv/ directory to git!

Add to .gitignore:

```
# .gitignore
venv/
env/
.venv/
*.pyc
__pycache__/
```

Pro Tip: Document Python version in README (e.g., Python 3.11)

venv Workflow: Team Member Installing venv

Steps for Team Member:

```
# 1. Clone the project repository
git clone https://github.com/team/project.git
cd project

# 2. Check Python version matches README
python --version

# 3. Create virtual environment
python -m venv venv

# 4. Activate environment
venv\Scripts\activate    # Windows

# 5. Install all dependencies from requirements.txt
pip install -r requirements.txt

# 6. Verify installation
pip list
# Ready to work!
```

Conda: The Data Science Powerhouse

What is Conda?

- Cross-platform package and environment manager
- Handles Python AND non-Python dependencies (C, R, Fortran)
- Pre-compiled binary packages for fast installation
- Advanced dependency resolution
- Channel system: defaults, conda-forge, bioconda

Installation: Download Anaconda or Miniconda from anaconda.com

When to use: Data science, ML, scientific computing, complex dependencies

Conda: Creating Environments

Creation Commands:

```
conda create -n myenv                          # Empty environment
# -n = --name

conda create -n myenv python=3.11              # Specify Python version

conda create -n dsenv python=3.11 numpy pandas  # With packages

conda create -n myenv python=3.11 -c conda-forge
# -c = --channel, use conda-forge
```

Tip: Use conda-forge channel for most up-to-date packages

Conda: Activation & Management

Essential Commands:

```
conda activate myenv           # Activate environment
conda deactivate               # Deactivate current env

conda env list                 # List all environments

conda remove -n myenv --all    # Delete environment
# -n = --name, --all = delete everything

conda update conda             # Update conda itself
```

Conda: Package Management

Package Commands:

```
conda install numpy                      # Install from default channel
conda install -c conda-forge pandas      # From conda-forge (-c = --channel)
conda install numpy=1.24.0               # Specific version
conda install "numpy>=1.24,<2.0"         # Version range
conda install numpy pandas matplotlib    # Multiple packages

conda update numpy                       # Update one package
conda update --all                       # Update all packages
conda remove numpy                       # Remove package

conda list                               # List installed packages
conda search numpy                       # Search for package
conda search "numpy>=1.24"               # Search with version
```

Conda: environment.yml Workflow

Creating environment.yml:

```
# Export current environment
conda env export > environment.yml

# Export only manually installed packages
conda env export --from-history > environment.yml
```

Creating from YAML:

```
# Create from file
conda env create -f environment.yml
```

Sample environment.yml:

```yaml
name: data_analysis
channels:
  - conda-forge
  - defaults
dependencies:
  - python=3.11
  - numpy>=1.24.0
  - pandas=2.0.*
  - matplotlib
  - scikit-learn
  - pip
  - pip:
    - streamlit
    - plotly
```

Conda Workflow: Creating & Using Conda Env

Complete Workflow:

```
# 1. Create environment with Python version
conda create -n myproject python=3.11

# 2. Activate environment
conda activate myproject

# 3. Install packages (prefer conda-forge)
conda install numpy pandas matplotlib -c conda-forge

# 4. Install additional packages as needed
conda install scikit-learn jupyter

# 5. Export environment
conda env export --from-history > environment.yml

# 6. Work on your project...

# 7. Deactivate when done
conda deactivate
```

Conda Workflow: Team Member Installing

Steps for Team Member:

```
# 1. Clone repository
git clone https://github.com/team/project.git
cd project

# 2. Create environment from YAML
conda env create -f environment.yml

# 3. Activate environment
conda activate data_analysis  # name from YAML file

# 4. Verify installation
conda list

# Ready to work!
```

Poetry: Modern Dependency Management

What is Poetry?

- Modern Python packaging and dependency management
- Deterministic builds with poetry.lock file
- Automatic virtual environment creation
- Dependency resolution with version constraints
- Built-in publishing to PyPI
- Single pyproject.toml configuration file
- Version 2.3.2 - Latest stable release → The one used in these slides

When to use: Package development, modern projects, team collaboration

Poetry: Installation & Project Setup (v2.3.2)

Installation Methods (Poetry 2.3.2):

```
# Pip installation
pip install poetry

# Verify version
poetry --version  # Should show: Poetry (version 2.3.2)
```

Creating New Project:

```
poetry new my-project
# Creates:
# my-project/
# ├── pyproject.toml
# ├── README.md
# ├── my_project/__init__.py
# └── tests/__init__.py
```

Initialize Existing:

```
cd existing-project
poetry init  # Interactive setup
# Guides you through creating
# pyproject.toml
```

Poetry: Managing Dependencies (v2.3.2)

Dependency Management Commands:

```
poetry add numpy                        # Add package (runtime)
poetry add 'numpy>=1.24,<2.0'           # Version constraints
poetry add numpy@latest                 # Add latest version

poetry remove numpy                     # Remove package

poetry update                           # Update all packages
poetry update numpy                     # Update specific package

poetry show                             # List dependencies

poetry install                          # Install all dependencies
```

Poetry: Virtual Environment Management

Poetry automatically creates and manages virtual environments!

Environment Commands:

```
poetry env use python3.11           # Set Python version
poetry env use /path/to/python      # Use specific Python

poetry env info                     # Show environment info
poetry env list                     # List environments
poetry env remove python3.11        # Delete environment

pip install poetry-plugin-shell     # Need to install this plugin to use shell
poetry shell                        # Spawn shell in venv

poetry run python script.py         # Run script in venv
poetry run pytest                   # Run tests in venv
```

Configuration:

```
poetry config virtualenvs.in-project true  # Create venv in project directory
```

Poetry: pyproject.toml Configuration (v2.3.2)

TOML Configuration:

```
[project]
name = "env-3-trials"
version = "0.1.0"
description = "Data Analysis"
authors = [
    {name = “myname",email =
“myemail@gmail.com"}
]
requires-python = ">=3.13"
dependencies = [
    "pendulum (>=3.2.0,<4.0.0)"
]

[build-system]
requires = ["poetry-core>=2.0.0,<3.0.0"]
build-backend = "poetry.core.masonry.api"
```

Version Constraints:

^1.24.0 → >=1.24.0, <2.0.0

~1.24.0 → >=1.24.0, <1.25.0

1.24.* → >=1.24.0, <1.25.0

>=1.20,<2.0 → Range

* → Any version

Poetry: The Lock File (poetry.lock)

Why Lock Files Matter:

- Guarantees exact same versions across all environments
- Prevents 'works on my machine' problems
- Records complete dependency tree (transitive dependencies)
- Automatically generated and updated by Poetry
- Should be committed to version control

Lock File Commands:

```
poetry lock                 # Update lock file
poetry install              # Install from lock file
poetry update numpy         # Update package + lock file
```

Poetry Workflow: Creating & Using Environment

Complete Workflow:

```
# 1. Create new project
poetry new myproject
cd myproject

# 2. Add dependencies
poetry add numpy pandas matplotlib

# 3. Install everything (creates venv automatically)
poetry install

# 4. Activate environment → shell needs to install: pip install poetry-
plugin-shell
poetry shell  # Or use 'poetry run' for commands

# 5. Work on project...
python my_code.py

# 6. Exit shell when done
exit
```

Poetry Workflow: Creating & Using Environment

Activate Environments Alternatives:

1- Using Shell (old way):

```
# 1. Activate environment → shell needs to install: pip install poetry-
plugin-shell
pip install poetry-plugin-shell
poetry shell  # Or use 'poetry run' for commands
```

2- Using Activate (new way):

```
# 1. Get the activation path
poetry env activate
"C:\Users\user\Desktop\env_trials\env_trials\.venv\Scripts\activate.bat"

# 2. Run that path, in Windows, copy and paste that entire path (including “”)
"C:\Users\user\Desktop\env_trials\env_trials\.venv\Scripts\activate.bat"
```

Poetry Workflow: Sharing with Team Members

What to Share:

- pyproject.toml (project configuration)
- poetry.lock (**exact** versions - CRITICAL!)
- README with setup instructions
- Your project source code
- DO NOT commit .venv/ directory!

Poetry Workflow: Team Member Installing

Steps for Team Member:

```
# 1. Clone repository
git clone https://github.com/team/project.git
cd project

# 2. Install Poetry and shell plugin if not already installed
pip install poetry poetry-plugin-shell

# 3. Install all dependencies from lock file
poetry install
# This creates venv and installs exact versions from poetry.lock

# 4. Activate environment
poetry shell

# 5. Verify installation
poetry show

# Ready to work!
```

Quick Reference Cheat Sheet

Essential Commands:

TASK	venv	Conda	Poetry
Create environment	python -m venv env	conda create -n env	poetry init
Activate	env\Scripts\activate	conda activate env	poetry shell
Install package	pip install pkg	conda install pkg	poetry add pkg
Install from file	pip install -r req.txt	conda env create -f e.yml	poetry install
Save dependencies	pip freeze > req.txt	conda env export > e.yml	poetry.lock (automatic)
Update package	pip install -U pkg	conda update pkg	poetry update pkg
Remove package	pip uninstall pkg	conda remove pkg	poetry remove pkg
List packages	pip list	conda list	poetry show
Deactivate	deactivate	conda deactivate	exit
Delete environment	rm -rf env/	conda env remove -n env	poetry env remove python

Official Documentation:

Poetry v2.3: www.python-poetry.org/docs/